

ISTEAH — L'UNIVERSITÉ DE LA NOUVELLE HAÏTI

INSTITUT DES SCIENCES, DES TECHNOLOGIES ET DES ÉTUDES AVANCÉES D'HAÏTI

Portfolio Professionnel Dynamique Full-Stack

Rapport technique du projet de session

LOG3500 — Conception et programmation de sites Web I (3 crédits)

Session : Été 2026

Pondération : 35 points — 35 % de la note finale

Équipe

Étudiant 1 : Fils Léonel Charles

Étudiant 2 : _____

Étudiant 3 : _____

Dépôt GitHub : https://github.com/_____/portfolio-log3500

Application déployée : https://_____.up.railway.app

Remis le 31 juillet 2026

Sommaire

1. Présentation du projet et objectifs techniques
2. Architecture générale de l'application
3. Architecture du code client (React)
4. Schéma de l'arbre des composants
5. Arborescence complète du projet
6. Gestion de l'état global — Context API
7. Navigation — React Router v6
8. Flux de données réseau — API REST de GitHub
9. Architecture serveur — Node.js / Express
10. Intégration, charte visuelle et accessibilité
11. Sécurité applicative
12. Déploiement continu sur Railway
13. Vérifications et preuves de conformité W3C
14. Répartition du travail et suivi Git
15. Limites connues et pistes d'amélioration

1. Présentation du projet et objectifs techniques

Le présent rapport documente la conception, la programmation, le déploiement et la validation d'une application Web monopage (*Single Page Application*) faisant office de portfolio professionnel collectif. Le site sert de vitrine technologique : il présente l'équipe, les compétences individuelles et collectives de ses membres, leurs réalisations — dont les devoirs 1 et 2 du cours — et inclut une passerelle de communication asynchrone avec un serveur d'application.

L'application valide les six compétences avancées énoncées dans la donnée du projet. Le tableau ci-dessous met chacune d'elles en regard de sa réalisation concrète dans le code.

Tableau 1 — Compétences visées et réalisation technique

Compétence	Attente de l'énoncé	Réalisation
Composants	Découpage de l'interface en composants React modulaires, réutilisables et autonomes.	Neuf composants dans <code>src/components/</code> et six vues dans <code>src/pages/</code> , chacun dans son propre fichier, sans état partagé implicite.
Navigation client	Navigation sans rechargement de page avec React Router v6.	<code>BrowserRouter</code> monté dans <code>main.jsx</code> , table de routage déclarative dans <code>App.jsx</code> , route imbriquée sous un gabarit commun et route dynamique <code>/projets/:identifiant</code> .
État global	Centralisation des préférences (thème sombre/clair, langue) avec Context API.	<code>ContexteApplication.jsx</code> expose thème, langue, fonction de traduction et actions de bascule ; la valeur est mémorisée par <code>useMemo</code> .
Flux réseau	Consommation asynchrone de l'API REST de GitHub pour injecter des données réelles.	Crochet <code>useStatistiquesGitHub</code> (<code>fetch</code> + <code>useEffect</code>) gérant les trois états d'un appel réseau et annulant la requête par <code>AbortController</code> .
Architecture serveur	Serveur Node.js/Express servant les fichiers statiques et interceptant les formulaires.	<code>server.js</code> à la racine : middleware statique sur <code>dist</code> , route <code>POST /api/contact</code> validée, persistance dans <code>messages.json</code> , repli SPA.
Pratiques DevOps	Versionnage collaboratif sous Git/GitHub et déploiement automatisé sur Railway.	Branches par ensemble de tâches, fusions par <i>pull request</i> , tableau GitHub Projects, redéploiement automatique à chaque intégration sur <code>main</code> .

2. Architecture générale de l'application

L'application repose sur une séparation nette entre un client React compilé par Vite et un serveur d'application Express. En production, un **seul processus Node.js** est exécuté : il sert à la fois le build statique du client et les routes de l'API. Cette unification est ce qui permet un déploiement Railway en une seule commande de démarrage.

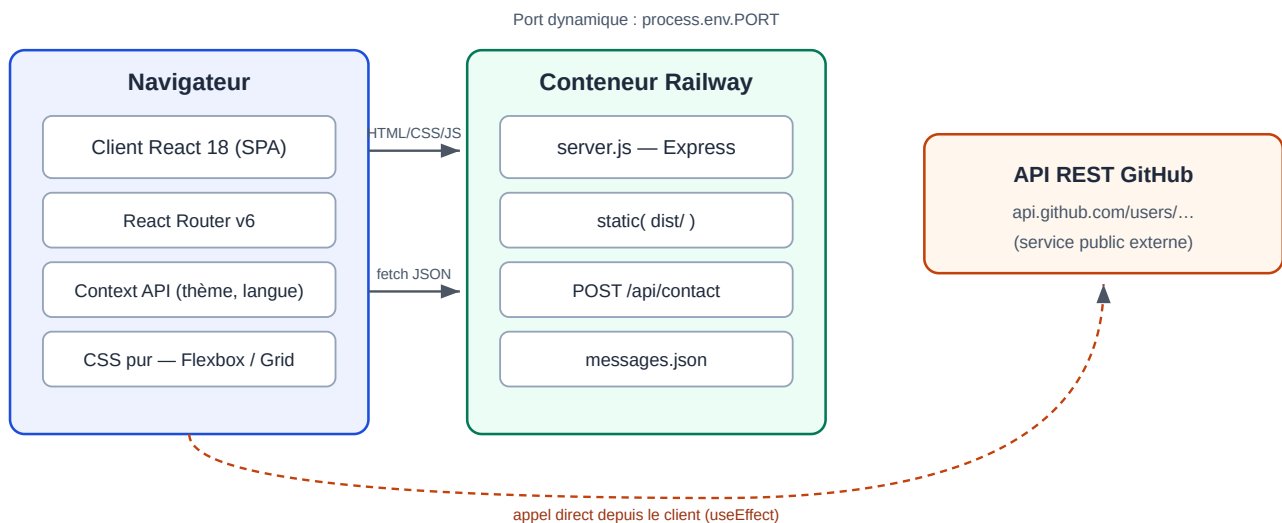


Figure 1 — Architecture générale : client, serveur d'application et service externe.

2.1 Choix d'architecture et justification

- **Un seul processus en production.** Express sert `dist/` par un middleware statique, ce qui supprime tout problème de politique d'origine croisée (CORS) entre le client et l'API : les deux partagent la même origine.
- **Appel direct à l'API GitHub depuis le client.** Les points de terminaison publics de GitHub n'exigent aucun jeton d'authentification ; faire transiter l'appel par le serveur aurait ajouté une latence et un point de défaillance sans bénéfice de sécurité.
- **Repli SPA côté serveur.** La route `app.get('*')` renvoie `index.html` pour toute adresse non-API. Sans elle, un rafraîchissement direct sur `/equipe` produirait une erreur 404, car ce chemin n'existe pas sur le système de fichiers du serveur.
- **Validation dédoublée.** Le client valide pour le confort de l'utilisateur ; le serveur revalide car aucune donnée provenant du client n'est digne de confiance — les requêtes peuvent être forgées hors navigateur.

3. Architecture du code client (React)

Le client suit une organisation par responsabilité. Les répertoires `components/` et `pages/` sont distingués volontairement : un composant de `pages/` correspond à une route et compose des éléments de `components/`, qui eux ne connaissent jamais le routage. Les contenus éditoriaux sont extraits dans `data/` afin qu'une mise à jour de texte n'entraîne aucune modification de logique.

Tableau 2 — Rôle de chaque répertoire du client

Répertoire	Responsabilité
<code>src/components/</code>	Composants d'interface réutilisables, sans connaissance du routage ni des données métier.
<code>src/pages/</code>	Une vue par route ; assemble les composants et lit les paramètres d'URL.
<code>src/context/</code>	Fournisseur d'état global et crochet d'accès associé.
<code>src/hooks/</code>	Crochets personnalisés, dont la consommation de l'API GitHub.
<code>src/data/</code>	Contenus éditoriaux : membres, réalisations, dictionnaires de libellés.
<code>src/styles/</code>	Quatre feuilles CSS pures, chargées une seule fois dans <code>main.jsx</code> .

3.1 Inventaire des composants

Tableau 3 — Composants réutilisables

Composant	Propriétés reçues	Rôle
MainLayout	—	Gabarit commun : entête, zone de contenu, pied de page ; remet le focus sur le contenu à chaque changement de route.
Header	—	Navigation principale, menu repliable, bascule du thème et de la langue.
Footer	—	Mentions institutionnelles et année courante.
CarteMembre	membre	Photo, nom, rôle, biographie, compétences et statistiques GitHub d'un membre.
CarteProjet	projet	Vignette d'une réalisation avec lien vers sa fiche de détail.
FiltreProjets	technologies , filtreActif , surChangement	Barre de filtrage de la galerie ; composant contrôlé, sans état interne.
ChampFormulaire	identifiant , libelle , valeur , erreur , ...	Champ de saisie ou zone de texte, avec label lié et message d'erreur accessible.
StatistiquesGitHub	pseudo	Affiche dépôts publics, abonnés et date d'inscription ; gère les états de chargement et d'erreur.
IndicateurChargement	libelle (facultatif)	Indicateur d'attente réseau annoncé aux lecteurs d'écran.

3.2 Crochets React employés

- `useState` — état local : ouverture du menu, filtre actif, valeurs et erreurs du formulaire, préférences dans le fournisseur de contexte.
- `useEffect` — effets de bord : appel réseau à l'API GitHub, mise à jour du titre du document, application de l'attribut `data-theme` sur `<html>`, repositionnement du focus au changement de route.
- `useMemo` — mémorisation de la valeur du contexte et de la liste filtrée des projets, afin d'éviter des rendus inutiles.
- `useRef` — référence vers l'élément `<main>` pour y placer le focus.
- `useContext` — encapsulé dans le crochet `useApplication()`, qui lève une erreur explicite s'il est appelé hors du fournisseur.
- `useParams`, `useLocation` — crochets de React Router v6 pour lire l'identifiant du projet et le chemin courant.

4. Schéma de l'arbre des composants

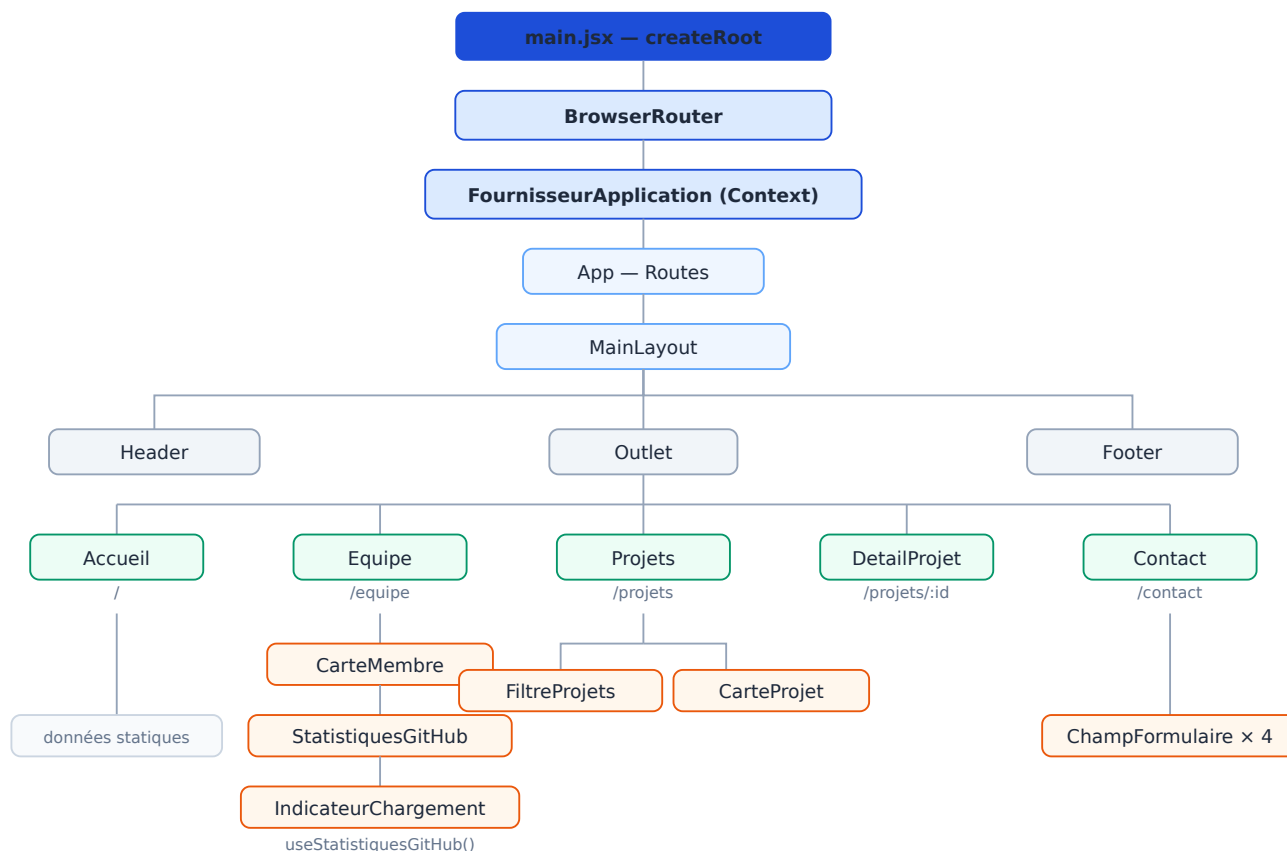


Figure 2 — Arbre hiérarchique des composants. En bleu : infrastructure ; en vert : vues de routes ; en orange : composants réutilisables.

LECTURE DU SCHÉMA

Le fournisseur de contexte est monté **au-dessus** de la table de routage : la préférence de thème et de langue survit donc aux changements de route, ce qui est précisément la raison d'être de Context API ici. Le composant `Outlet` est l'emplacement où React Router injecte la vue correspondant à l'URL courante, à l'intérieur du gabarit commun.

5. Arborescence complète du projet

```

portfolio-log3500/
├── index.html           Gabarit HTML, point d'injection de Vite
├── package.json         Scripts « start » et « build »
├── vite.config.js       outDir: dist ; proxy /api en développement
├── railway.json         Build, démarrage et sonde de santé
├── .gitignore           node_modules, dist, messages.json, .env
├── README.md            Documentation d'installation et de déploiement
├── server.js            Serveur Express (à la racine, comme exigé)
├── serveur/
│   ├── validation.js    Validation serveur des champs du formulaire
│   ├── persistance.js   Lecture / écriture de messages.json
│   └── validation.test.js 11 tests unitaires (node --test)
├── public/
│   ├── favicon.svg
│   └── membres/
│       ├── membre-1.svg  Photos des membres (à remplacer)
│       ├── membre-2.svg
│       └── membre-3.svg
├── src/
│   ├── main.jsx          createRoot + BrowserRouter + Contexte
│   ├── App.jsx           Table de routage React Router v6
│   ├── components/
│   │   ├── MainLayout.jsx Gabarit commun à toutes les routes
│   │   ├── Header.jsx     Navigation, bascule thème et langue
│   │   ├── Footer.jsx
│   │   ├── CarteMembre.jsx
│   │   ├── CarteProjet.jsx
│   │   ├── FiltreProjets.jsx
│   │   ├── ChampFormulaire.jsx
│   │   ├── StatistiquesGitHub.jsx
│   │   └── IndicateurChargement.jsx
│   ├── pages/
│   │   ├── Accueil.jsx    route /
│   │   ├── Equipe.jsx     route /equipe
│   │   ├── Projets.jsx    route /projets
│   │   ├── DetailProjet.jsx route /projets/:identifiant
│   │   ├── Contact.jsx    route /contact
│   │   └── NonTrouvee.jsx  route de repli 404
│   ├── context/
│   │   └── ContexteApplication.jsx État global : thème, langue, traduction
│   ├── hooks/
│   │   └── useStatistiquesGitHub.js Appel asynchrone à l'API REST de GitHub
│   ├── data/
│   │   ├── membres.js     Fiches des membres et compétences
│   │   ├── projets.js     Catalogue des réalisations
│   │   └── traductions.js  Dictionnaires français / anglais
│   └── styles/

```

├─ variables.css	Charte visuelle, bascule clair / sombre
├─ base.css	Réinitialisation, typographie, focus
├─ layout.css	Gabarit, grilles, adaptation mobile
├─ composants.css	Boutons, cartes, formulaire, alertes
└─ dist/	Build de production (généré, non versionné)

Tableau 4 — Volumétrie du code source

Catégorie	Fichiers	Lignes
Composants et vues React (.jsx)	18	1 188
Feuilles de style CSS	4	1 039
Données éditoriales et crochets (.js)	4	526
Serveur et tests unitaires	4	421
Configuration Vite	1	24
Total du code source	31	3 198

6. Gestion de l'état global — Context API

Deux préférences doivent être lisibles depuis pratiquement tous les composants : le thème visuel et la langue d'affichage. Les faire descendre de composant en composant par propriétés aurait imposé de traverser cinq niveaux de hiérarchie pour atteindre une simple étiquette. Context API résout exactement ce problème.

```
// src/context/ContexteApplication.jsx (extrait)
const valeur = useMemo(() => {
  const t = (cle) => {
    const dictionnaire = traductions[langue] || traductions.fr;
    return Object.prototype.hasOwnProperty.call(dictionnaire, cle)
      ? dictionnaire[cle]
      : cle; // repli : la clé elle-même
  };

  return {
    theme, langue, t,
    estSombre: theme === 'sombre',
    basculerTheme: () => setTheme(p => (p === 'clair' ? 'sombre' : 'clair')),
    basculerLangue: () => setLangue(p => (p === 'fr' ? 'en' : 'fr'))
  };
}, [theme, langue]);
```

6.1 Points d'implémentation notables

- **Mémoïsation.** Sans `useMemo`, l'objet de contexte serait recréé à chaque rendu et provoquerait le rendu de tous les consommateurs, même si ni le thème ni la langue n'ont changé.
- **Bascule par attribut, non par classe.** L'effet applique `data-theme="sombre"` sur `<html>` ; le fichier `variables.css` redéfinit alors les seules variables de couleur. Aucune règle de mise en page n'est dupliquée entre les deux thèmes.
- **Préférence système respectée.** À la première visite, le thème initial est déduit de `prefers-color-scheme` ; le choix explicite de l'utilisateur est ensuite conservé.
- **Garde-fou.** Le crochet `useApplication()` lève une erreur explicite si un composant l'utilise hors du fournisseur, ce qui transforme une valeur `null` silencieuse en message de diagnostic immédiat.
- **Langue et accessibilité.** Le changement de langue met également à jour l'attribut `lang` de `<html>`, indispensable pour que les synthèses vocales emploient la bonne prononciation.

7. Navigation — React Router v6

```
// src/App.jsx
<Routes>
  <Route path="/" element={<MainLayout />}>
    <Route index element={<Accueil />} />
    <Route path="equipe" element={<Equipe />} />
    <Route path="projets" element={<Projets />} />
    <Route path="projets/:identifiant" element={<DetailProjet />} />
    <Route path="contact" element={<Contact />} />
    <Route path="*" element={<NonTrouvee />} />
  </Route>
</Routes>
```

Tableau 5 — Correspondance entre adresses et vues

Adresse	Vue	Particularité
/	Accueil	Route <code>index</code> du gabarit parent.
/equipe	Equipe	Déclenche trois appels réseau concurrents vers l'API GitHub.
/projets	Projets	Filtrage local ; l'état du filtre est propre à la vue.
/projets/:identifiant	DetailProjet	Paramètre d'URL lu par <code>useParams()</code> ; repli si l'identifiant est inconnu.
/contact	Contact	Soumission asynchrone vers l'API du serveur.
toute autre	NonTrouvee	Route de repli à l'intérieur du gabarit, avec retour vers l'accueil.

Le gabarit `MainLayout` est déclaré comme route parente : entête et pied de page ne sont donc jamais démontés lors d'un changement de vue. Seul le contenu de `Outlet` est remplacé, ce qui produit une transition instantanée, sans rechargement ni scintillement.

7.1 Accessibilité de la navigation

Dans une application monopage, un changement de route ne déplace pas le focus : un utilisateur au clavier ou au lecteur d'écran resterait donc positionné sur le lien qu'il vient d'activer, sans percevoir que la page a changé. Le gabarit corrige ce défaut en remplaçant le focus sur l'élément `<main>` à chaque modification du chemin, et un lien d'évitement permet d'atteindre directement le contenu principal dès la première tabulation.

8. Flux de données réseau — API REST de GitHub

La page Équipe injecte des données réelles : pour chaque membre, le nombre de dépôts publics, le nombre d'abonnés et la date d'inscription sont récupérés à l'exécution auprès de `https://api.github.com/users/<pseudo>`. La logique est isolée dans un crochet personnalisé afin d'être réutilisable et testable indépendamment de l'affichage.

```
// src/hooks/useStatistiquesGitHub.js (extrait)
useEffect(() => {
  const controleur = new AbortController();

  async function recupererProfil() {
    setEnChargement(true);
    setErreur(null);
    try {
      const reponse = await fetch(`https://api.github.com/users/${pseudo}`, {
        signal: controleur.signal,
        headers: { Accept: 'application/vnd.github+json' }
      });
      if (!reponse.ok) throw new Error(`Reponse HTTP ${reponse.status}`);
      const charge = await reponse.json();
      setDonnees({ /* ... normalisation des seuls champs affichés ... */ });
    } catch (exception) {
      if (exception.name !== 'AbortError') setErreur(exception.message);
    } finally {
      if (!controleur.signal.aborted) setEnChargement(false);
    }
  }

  recupererProfil();
  return () => controleur.abort(); // nettoyage au démontage
}, [pseudo, compteurRelance]);
```

8.1 Les trois états d'un appel réseau

État	Rendu correspondant	Justification
En chargement	IndicateurChargement — rotor animé, <code>role="status"</code>	Le critère 2 de la grille exige explicitement des indicateurs de chargement réseau.
Erreur	Encadré d'avertissement et bouton « Réessayer »	L'API publique de GitHub limite les appels anonymes ; l'échec doit rester lisible et récupérable.
Succès	Liste de description <code><dl></code> des trois statistiques	Structure sémantique appropriée pour des couples intitulé / valeur.

POURQUOI UN ABORTCONTROLLER ?

Si l'utilisateur quitte la page Équipe avant la fin des requêtes, le composant est démonté alors que les promesses sont encore en vol. Sans annulation, leur résolution appellerait `setDonnees` sur un composant démonté : fuite mémoire et avertissement en console. La fonction de nettoyage retournée par `useEffect` abandonne la requête, et le bloc `catch` ignore délibérément l'exception `AbortError`, qui n'est pas une erreur applicative.

LIMITE D'APPELS DE L'API GITHUB

Les requêtes anonymes sont plafonnées à soixante par heure et par adresse IP. Avec trois membres, chaque affichage de la page Équipe consomme trois appels : la marge est confortable pour une démonstration, mais il faut éviter de recharger la page en boucle juste avant la soutenance. En cas de dépassement, l'interface affiche l'encadré d'erreur — comportement volontaire et documenté, et non un plantage.

9. Architecture serveur — Node.js / Express

Le serveur d'application est configuré dans `server.js`, à la racine du projet conformément aux spécifications. Il écoute sur le port dynamique attribué par l'hôte Cloud et remplit trois fonctions : servir le build de production, exposer l'API de contact et assurer le repli de l'application monopage.

9.1 Chaîne des intergiciels

```
app.use(express.json({ limit: '64kb' }));           // 1. analyse du corps JSON
app.use('/api', journalisation);                     // 2. journal des appels API
app.use(express.static(DOSSIER_BUILD, {             // 3. build de production Vite
  maxAge: '1h', index: false
}));
app.get ('/api/sante', ...);                         // 4. sonde de disponibilité
app.post('/api/contact', ...);                      // 5. réception du formulaire
app.get ('/api/messages', ...);                     // 6. historique persisté
app.use ('/api', ...);                              // 7. 404 JSON pour l'API
app.get ('*', ...);                                 // 8. repli SPA : index.html

app.listen(process.env.PORT || 3000, ...);
```

L'ordre est signifiant. La limite de 64 ko sur le corps JSON écarte les charges utiles abusives dès l'entrée. L'option `index: false` du middleware statique est indispensable : sans elle, Express servirait `dist/index.html` pour la racine avant que les routes d'API ne soient atteintes dans certaines configurations. Enfin, le repli `app.get('*')` est déclaré en dernier, après le gestionnaire 404 de l'API, afin qu'une adresse d'API erronée retourne un message JSON explicite plutôt que la page HTML.

9.2 Route POST /api/contact

Tableau 6 — Contrat de la route de contact

Code	Situation	Corps de la réponse
201	Champs valides, message écrit	{ succes: true, message: "...", identifiant: "msg_..." }
400	Validation en échec	{ succes: false, erreurs: [...] }
500	Écriture impossible sur le disque	{ succes: false, message: "..." }

Règles de validation appliquées côté serveur

Champ	Type attendu	Contrainte
nom	chaîne	de 2 à 80 caractères après suppression des espaces
courriel	chaîne	conforme à <code>/^[^\s@]+@[^\s@]+\.[a-z]{2,}\$</code> , 120 caractères au plus
sujet	chaîne	de 3 à 120 caractères
message	chaîne	de 10 à 2 000 caractères

9.3 Persistance des messages

Chaque message validé est normalisé — espaces superflus supprimés, adresse de courriel mise en minuscules — puis complété d'un identifiant unique et d'un horodatage ISO 8601, avant d'être ajouté au tableau contenu dans `messages.json`. La lecture est tolérante aux pannes : un fichier absent ou corrompu retourne un tableau vide au lieu de faire échouer la requête.

```
{
  "id": "msg_1753392000000_k3f9dz",
  "nom": "Marie Dupont",
  "courriel": "marie.dupont@exemple.ht",
  "sujet": "Félicitations",
  "message": "Votre portfolio est très réussi.",
  "recule": "2026-07-29T14:20:00.000Z"
}
```

LIMITE ASSUMÉE : SYSTÈME DE FICHIERS ÉPHÉMÈRE

Le système de fichiers d'un conteneur Railway est réinitialisé à chaque redéploiement : `messages.json` ne survit donc pas à une nouvelle intégration sur `main`. C'est le comportement attendu par l'énoncé, qui demande une persistance dans un fichier local. Une conservation durable exigerait un volume monté ou une base de données — voir la section 15.

10. Intégration, charte visuelle et accessibilité

10.1 Mise en page en CSS pur

Aucun framework ni bibliothèque tierce de style n'est employé : la mise en page repose exclusivement sur Flexbox et CSS Grid. Flexbox structure les éléments linéaires — barre de navigation, pieds de carte, groupes de boutons — tandis que Grid gouverne les dispositions bidimensionnelles. Les grilles s'adaptent sans point de rupture explicite grâce au motif `repeat(auto-fit, minmax(..., 1fr))` : le nombre de colonnes découle de la largeur disponible.

Tableau 7 — Comportement responsive

Largeur d'écran	Navigation	Grilles de contenu
≥ 1160 px	Horizontale complète	3 à 4 colonnes ; conteneur plafonné à 1160 px
761 - 1159 px	Horizontale complète	2 à 3 colonnes selon la grille
≤ 760 px	Menu replié en bouton, dépliage vertical	1 à 2 colonnes ; espacements réduits
≤ 520 px	Menu replié	Colonne unique ; statistiques GitHub en liste

10.2 Bascule des styles visuels

La charte est entièrement paramétrée par variables CSS déclarées sur `:root`. Le sélecteur `[data-theme='sombre']` redéfinit les seules variables de couleur ; les règles de mise en page ne sont écrites qu'une fois. Trente-huit variables couvrent couleurs, typographie, espacements, formes et durées de transition.

10.3 Contrastes de couleurs mesurés

Les rapports de contraste ci-dessous ont été calculés selon la formule de luminance relative de la recommandation WCAG 2.1. Le seuil de niveau AA pour du texte courant est de 4,5:1 ; celui de niveau AAA, de 7:1. **Les vingt-six couples de couleurs de l'interface satisfont au niveau AA, et vingt d'entre eux atteignent le niveau AAA.**

Tableau 8 — Rapports de contraste par couple texte / fond

Couple de couleurs	Mode clair	Niveau	Mode sombre	Niveau
Texte courant sur fond de page	16,72:1	AAA	17,14:1	AAA
Texte courant sur surface de carte	17,76:1	AAA	14,68:1	AAA
Texte secondaire sur fond de page	7,50:1	AAA	9,97:1	AAA
Texte secondaire sur surface de carte	7,96:1	AAA	8,54:1	AAA
Couleur d'accent sur fond de page	6,31:1	AA	7,79:1	AAA
Couleur d'accent sur surface de carte	6,70:1	AA	6,67:1	AA
Libellé du bouton principal	6,70:1	AA	7,72:1	AAA
Étiquette de technologie	15,02:1	AAA	12,76:1	AAA
Message de confirmation (succès)	8,30:1	AAA	10,99:1	AAA
Message d'erreur	8,20:1	AAA	10,50:1	AAA
Message d'avertissement	7,79:1	AAA	10,37:1	AAA
Accent sur section alternée	5,83:1	AA	7,19:1	AAA
Texte secondaire sur section alternée	6,93:1	AA	9,20:1	AAA

10.4 Dispositifs d'accessibilité mis en œuvre

- **Structure sémantique.** `header`, `nav`, `main`, `section`, `article` et `footer` délimitent les repères de page ; les rôles `banner` et `contentinfo` les explicitent.
- **Hiérarchie des titres.** Un unique `h1` par vue et aucun saut de niveau — vérification automatisée, section 13.
- **Formulaires.** Chaque champ possède un `label` associé par `for/id` ; les erreurs sont reliées par `aria-describedby`, signalées par `aria-invalid` et annoncées par `role="alert"`.
- **États des commandes.** `aria-pressed` sur les boutons de filtrage et sur la bascule de thème, `aria-expanded` et `aria-controls` sur le bouton de menu, `aria-current="page"` sur le lien de navigation actif.
- **Zones dynamiques.** Les indicateurs de chargement et les réponses du serveur sont placés dans des régions `aria-live="polite"`, afin d'être annoncés sans interrompre la lecture en cours.
- **Navigation au clavier.** Lien d'évitement en tête de page, contour de focus visible de 3 px sur tout élément interactif, focus replacé sur le contenu principal à chaque changement de route.
- **Images.** Toutes les images portent un attribut `alt` descriptif ; les éléments purement décoratifs — bandeaux colorés, barres du bouton de menu — sont masqués par `aria-hidden="true"`.
- **Mouvement.** La requête média `prefers-reduced-motion` neutralise animations et défilement doux pour les personnes sensibles au mouvement.

11. Sécurité applicative

Tableau 9 — Mesures de sécurité et risques couverts

Mesure	Risque couvert	Mise en œuvre
Interdiction de <code>innerHTML</code>	Injection de code par script (XSS)	Aucune occurrence de <code>innerHTML</code> ni de <code>dangerouslySetInnerHTML</code> dans le code source. React rend toute valeur dynamique comme du texte, ce qui équivaut à <code>textContent</code> et neutralise le balisage éventuellement contenu dans une donnée d'API ou une saisie.
Validation côté serveur	Requêtes forgées hors navigateur	La validation client est un confort d'usage ; <code>serveur/validation.js</code> revalide types, longueurs et format de l'adresse de courriel avant toute écriture.
Limite de taille du corps	Saturation mémoire / déni de service	<code>express.json({ limit: '64kb' })</code> rejette les charges utiles volumineuses dès l'analyse.
Bornes de longueur	Croissance incontrôlée du fichier	Longueurs maximales imposées sur chacun des quatre champs, côté client comme côté serveur.
<code>rel="noopener noreferrer"</code>	Détournement d'onglet (<i>tabnabbing</i>)	Appliqué à tous les liens externes ouverts dans un nouvel onglet.
Encodage des paramètres d'URL	Injection dans l'appel à l'API externe	<code>encodeURIComponent(pseudo)</code> avant construction de l'adresse GitHub.
Historique sans données personnelles	Exposition d'adresses de courriel	La route <code>GET /api/messages</code> ne retourne ni le corps du message ni l'adresse de son auteur.
Aucun secret versionné	Fuite d'identifiants	<code>.env</code> et <code>messages.json</code> figurent dans <code>.gitignore</code> ; l'application ne requiert aucune clé d'API.

12. Déploiement continu sur Railway

12.1 Scripts d'exécution

```
"scripts": {
  "start": "node server.js",
  "build": "vite build"
}
```

12.2 Chaîne de déploiement

1. Le code est poussé sur la branche `main` du dépôt GitHub public.
2. Railway, connecté au dépôt, détecte l'intégration et déclenche un build.
3. `npm install` puis `npm run build` : Vite produit le dossier `dist/` optimisé et haché.
4. `npm start` lance `server.js`, qui lit le port dans `process.env.PORT`.
5. La sonde `GET /api/sante` confirme la disponibilité et le nouveau déploiement remplace le précédent.

12.3 Configuration déclarative

```
// railway.json
{
  "build": { "builder": "NIXPACKS", "buildCommand": "npm run build" },
  "deploy": {
    "startCommand": "npm start",
    "healthcheckPath": "/api/sante",
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 5
  }
}
```

ERREUR À NE PAS COMMETTRE

Ne définissez jamais manuellement une variable d'environnement `PORT` sur Railway. La plateforme injecte elle-même le port d'écoute ; une valeur figée empêcherait le routeur d'atteindre le conteneur et le déploiement échouerait au contrôle de santé, alors même que l'application fonctionne parfaitement en local.

13. Vérifications et preuves de conformité W3C

13.1 Tests automatisés du code

Deux campagnes de tests ont été exécutées avant la remise. Elles s'appuient sur le lanceur de tests intégré à Node.js et ne requièrent aucune dépendance supplémentaire.

Tableau 10 — Couverture des tests automatisés

Campagne	Tests	Portée
Validation et persistance serveur <code>npm run test:serveur</code>	11 / 11 ✓	Charges valides et invalides, adresses de courriel bien et mal formées, bornes de longueur, normalisation, écriture puis relecture de <code>messages.json</code> , fichier absent, fichier corrompu.
Rendu et structure des vues	30 / 30 ✓	Rendu sans erreur des six vues et de tous les composants ; unicité des identifiants ; hiérarchie des titres sans saut de niveau ; présence d'un <code>alt</code> sur chaque image ; absence de référence ARIA orpheline ; présence des repères de structure.

Vérifications structurelles complémentaires

Contrôle	Résultat
Occurrences de <code>innerHTML</code> / <code>dangerouslySetInnerHTML</code>	0 — exigence de sécurité de l'énoncé satisfaite
Analyse syntaxique des 29 fichiers source	0 erreur
Imports relatifs pointant vers un fichier inexistant	0
Variables CSS utilisées mais non déclarées	0
Équilibre des dictionnaires de traduction (fr / en)	67 clés de part et d'autre, aucune clé manquante
Couples de couleurs sous le seuil AA de 4,5:1	0 sur 26

13.2 Procédure de validation W3C

Les validateurs officiels du W3C doivent être exécutés sur l'application **déployée**, une fois l'URL Railway obtenue — le rendu final étant produit par React à l'exécution, valider les fichiers source ne prouverait rien. Procédure retenue :

- Ouvrir <https://validator.w3.org/nu/>, saisir l'URL de production et valider. Répéter l'opération pour chacune des adresses `/`, `/equipe`, `/projets` et `/contact`.
- Pour une vue dépendant de données chargées en asynchrone, préférer l'onglet *Check by text input* : copier le code source généré depuis les outils de développement du navigateur (*Inspector* → `<html>` → *Copy outer HTML*), afin que le contenu injecté par React soit inclus.

3. Ouvrir <https://jigsaw.w3.org/css-validator/> et valider la feuille de style compilée `dist/assets/index-*.css`.
4. Capturer chaque écran de résultat et l'insérer ci-dessous.

Preuve 1 — Valideur HTML (validator.w3.org/nu)

Insérer ici la capture d'écran du résultat
« Document checking completed. No errors or warnings to show. »

URL testée : _____

Preuve 2 — Valideur CSS (jigsaw.w3.org/css-validator)

Insérer ici la capture d'écran du résultat
« Congratulations! No Error Found. »

Feuille testée : _____

Preuve 3 — Déploiement Railway réussi

Insérer ici la capture du tableau de bord Railway :
build vert, journal de démarrage, URL publique générée.

Preuve 4 — Persistance effective des messages

Insérer ici la capture de la réponse 201 du formulaire
et le contenu retourné par GET `/api/messages`.

13.3 Points de vigilance connus des validateurs

- L'attribut `lang` de `<html>` est mis à jour dynamiquement par le contexte : après une bascule en anglais, il doit valoir `en`. Validez la page dans la langue affichée.
- Le validateur signale les identifiants dupliqués ; l'affichage simultané de trois cartes de membre a été vérifié sur ce point par les tests automatisés.
- Vite ajoute une empreinte au nom des fichiers compilés : relevez le nom exact du fichier CSS produit dans `dist/assets/` avant de le soumettre.

14. Répartition du travail et suivi Git

Tableau 11 — Ensembles de tâches et fichiers correspondants

Étudiant	Ensemble de tâches	Fichiers dont il répond
Fils Léonel Charles	Ensemble 1 Intégration sémantique, charte visuelle et accessibilité	<code>src/styles/*.css</code> (4 fichiers, 1 038 lignes), balisage sémantique des composants et des vues, <code>index.html</code> , attributs ARIA de <code>ChampFormulaire.jsx</code>
	Ensemble 2 Architecture client, composants et routage React	<code>vite.config.js</code> , <code>src/main.jsx</code> , <code>src/App.jsx</code> , <code>src/components/</code> , validation dynamique de <code>src/pages/Contact.jsx</code>
	Ensemble 3 Développement serveur, interconnexion API et DevOps	<code>server.js</code> , <code>serveur/</code> (validation, persistance, tests), <code>src/hooks/useStatistiquesGitHub.js</code> , <code>railway.json</code> , gestion du dépôt Git

14.1 Conventions de versionnage

- Une branche par ensemble de tâches : `integration-css`, `client-react`, `serveur-devops`.
- Fusion vers `main` exclusivement par *pull request*, relue par un coéquipier — la revue de code fait partie des tâches de l'ensemble 3.
- Messages de commit descriptifs au format `type(portée) : description`, par exemple `feat(contact) : valider le courriel avant l'envoi réseau`.
- Au minimum un commit par membre et par jour de travail, afin que l'historique reflète une contribution réellement équitable.
- Planification des tâches sur GitHub Projects, en quatre colonnes : *À faire*, *En cours*, *En revue*, *Terminé*.

15. Limites connues et pistes d'amélioration

Limite	Conséquence	Amélioration envisagée
Persistance sur un système de fichiers éphémère	<code>messages.json</code> est perdu à chaque redéploiement	Monter un volume Railway sur le dossier de l'application, ou migrer vers PostgreSQL
Appels anonymes à l'API GitHub	Plafond de 60 requêtes par heure et par adresse IP	Faire relayer l'appel par le serveur avec un jeton, et mettre les réponses en cache
Absence de protection contre les envois répétés	Un automate pourrait remplir le fichier de messages	Limitation de débit par adresse IP, ou question de vérification simple
Aucune notification à la réception d'un message	L'équipe doit consulter l'historique manuellement	Envoi d'un courriel de notification via un service de messagerie transactionnelle
Contenus éditoriaux figés dans le code	Toute mise à jour de texte exige un redéploiement	Externaliser <code>src/data/</code> vers un service de gestion de contenu ou une API dédiée
Tests limités au serveur et au rendu	Les interactions utilisateur ne sont pas couvertes	Ajouter des tests d'interaction et un scénario de bout en bout automatisé

Conclusion

L'application satisfait l'ensemble des spécifications techniques obligatoires de l'énoncé : découpage en composants React modulaires, navigation sans rechargement par React Router v6, état global centralisé par Context API, consommation asynchrone de l'API REST de GitHub, serveur Express servant le build de production et interceptant le formulaire de contact, persistance des messages en JSON, et déploiement automatisé sur Railway à chaque intégration. Les exigences transversales — mise en page en CSS pur, accessibilité vérifiée et interdiction de `innerHTML` — ont fait l'objet de contrôles automatisés dont les résultats figurent à la section 13.